



Тестабилни дизајн, NUnit, Moq и покривеност кода

Тестови као доказ пословног понашања и квалитета граница

Тестабилни дизајн се пројектује

Unit тестирање није активност која се накнадно додаје готовом коду.

Раздвајање одговорности, јасни уговори и dependency injection омогућавају да се компоненте проверавају изоловано и детерминистички.

КЉУЧНА ПОРУКА

Тестови су повратна информација о квалитету граница система.

Test Explorer се чита као развојни сигнал

Један неуспех усмерава дијагностику. Укупан пролаз не замењује проверу захтева.

Test Explorer

ERS.Tests

- ✓ Reserve_WhenFree_ReturnsSuccess
- ✓ Reserve_WhenOverlap_ReturnsConflict
- ✓ Reserve_WhenInactive_ReturnsFailure
- ✗ Cancel_AfterDeadline_ReturnsFailure

3 теста су прошла, један захтева анализу.

Како се чита резултат

Црвени тест прво треба разумети, па тек онда мењати код.

Cancel_AfterDeadline_ReturnsFailure

Очекивано: CancellationWindowExpired

Добијено: Success

Тумачење

Порука показује да ли је проблем у правилу, имплементацији или очекивању теста. Код се мења тек после разумевања узрока.

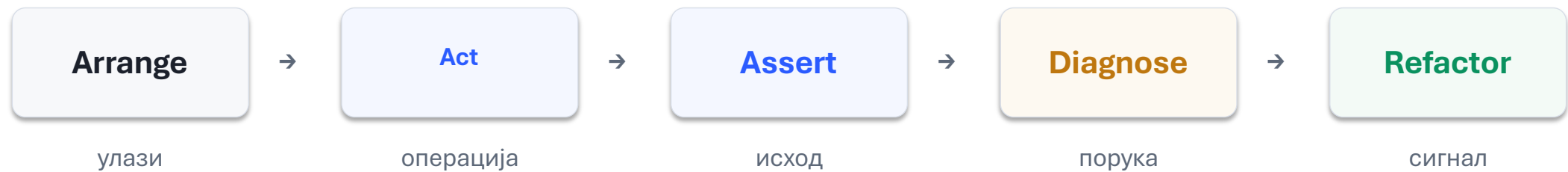
ПРОШЛО 3

ПАЛО 1

ТРАЈАЊЕ 184 ms

AAA структура чини тест читљивим

Један тест треба да има један јасан разлог постојања.



КЉУЧНА ПОРУКА

Назив теста треба да описује понашање које штити, а не само методу коју позива.

Негативни сценарио из критеријума прихватања

Тест проверава понашање, не приватне детаље имплементације.

```
1 [Test]
2 public void Reserve_WhenPeriodOverlaps_ReturnsConflict()
3 {
4     var repository = new Mock<IReservationRepository>();
5     repository.Setup(r => r.HasOverlap(It.IsAny<Guid>(), It.
6         IsAny<DateTime>(), It.IsAny<DateTime>()))
7         .Returns(true);
8
9     var result = service.Reserve(TestRequests.
10         ValidFutureReservation());
11
12     Assert.That(result.Success, Is.False);
13     Assert.That(result.Error, Is.EqualTo("OverlappingReservat
14         ion"));
15 }
```

Шта је важно

Arrange припрема контролисаног сарадника.

Act извршава једну релевантну операцију.

Assert проверава посматрани исход.

Test double није замена за сваки објекат

Mock се користи на граници коју тест намерно изолије.

Мали value object или доменска класа чије понашање је предмет теста најчешће треба да буде стварни објекат, не mock.

КЉУЧНА ПОРУКА

Mock је користан за repository, HTTP клијент, време или случајност.

Када користити mock

Изолација има смисла само када контролише променљиву спољну зависност.

Ситуација	Одлука
Repository или HTTP клијент	mock контролише повратну вредност
IClock или generator ID	mock обезбеђује детерминизам
Мали value object	користити стварни објекат
Доменска класа која се тестира	не mock-овати предмет теста

Moq Setup и Verify

Setup контролише сарадника, Verify проверава значајну интеракцију.

```
1 var repository = new Mock<IReservationRepository>();
2 repository.Setup(r => r.FindById(reservationId))
3     .Returns(existingReservation);
4
5 var result = service.Cancel(reservationId);
6
7 repository.Verify(r => r.Save(existingReservation), Times.
8     Once);
```

Шта је важно

Verify је оправдан када је интеракција део понашања.

Превише Verify провера прави крхак тест.

Не проверавати сваки интерни позив без потребе.

Тест сценарији потичу из правила

Највреднији тестови штите пословни ризик.

01

Успешан ток

валидни подаци и очекиван
резултат

02

Негативни ток

очекивано одбијање
захтева

03

Гранични случај

вредност која мења
понашање

План тестирања за резервисање

Пре кода треба знати шта се доказује.

Врста	Пример
Успешан сценарио	опрема је слободна, корисник овлашћен, термин валидан
Негативни сценарио	термин се преклапа са постојећом резервацијом
Негативни сценарио	опрема је неактивна
Гранични сценарио	нови термин почиње тачно када се претходни завршава
Инфраструктурни квар	repository пријављује грешку

Покретање тестова и coverage сигнала

Покривеност показује шта је извршено, не да ли је провера смислена.



```
1 dotnet test
2
3 # ако је collector подешен
4 dotnet test --collect:"XPlat Code Coverage"
```

Шта је важно

Најпре дефинисати пословно важне сценарије.

Coverage користити као дијагностички сигнал.

Висок проценат не доказује одсуство грешака.

Покривеност кода није циљ

Није корисно тестирати тривијалне гетере ако важна програмска грана остаје непроверена.

Питање није „колики је проценат”, већ који непокривени део носи највећи ризик и који тест би дао највише нових информација.

КЉУЧНА ПОРУКА

То је професионална употреба coverage извештаја.

Шта coverage извештај треба да покрене

Извештај је почетак анализе, не завршна оцена.

01

Условне гране

да ли су оба исхода проверена

02

Обрада грешака

да ли грешка не постаје лажни успех

03

Критично стање

новац, дозволе, резервације

04

Непокривен ризик

образложити или покрити тестом

Р3 граница

До ове тачке језгро мора бити ручно разумљиво и тестирано.

manual-core-baseline

стабилна верзија

После ове контролне тачке AI добија већу улогу, али систем већ има структуру и тестове који омогућавају независну проверу.



P3 — manual-core-baseline

Контролна тачка раздваја ручно пројектовано језгро од почетка AI фазе.

- 01 Кључни use-case-ови имају NUnit тестове.
- 02 Моq се користи само за променљиве спољне зависности.
- 03 Coverage је прегледан и ризична грана је покривена или образложена.
- 04 Најмање један bug је прво репродукован тестом.
- 05 Стабилна верзија језгра означена је Git tag-ом manual-core-baseline.

Добар тест је стабилан на refactoring

Ако тест пада при промени приватне имплементације без промене понашања, вероватно је превише крхак.

ТЕСТИРАТИ

видљиви резултат

стабилан код неуспеха

значајну интеракцију

гранично понашање

НЕ ВЕЗИВАТИ СЕ ЗА

приватну методу

редослед непотребних позива

тренутну структуру петље

тривијалне setter-e

Bug fix треба да почне тестом

Тест прво репродукује проблем, онда промена поправља понашање.

01

Reproduce

тест показује тренутни bug

02

Fix

минимална измена
понашања

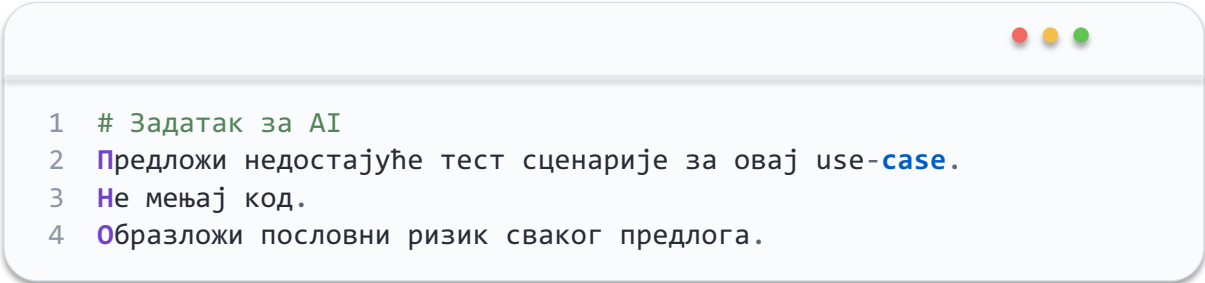
03

Protect

тест остаје као регресиона
заштита

Мини домаћи: AI предлаже сценарије

AI може помоћи у откривању пропуштених тестова, али тим одлучује.

- 
- ```
1 # Задатак за AI
2 Предложи недостајуће тест сценарије за овај use-case.
3 Не мењај код.
4 Образложи пословни ризик сваког предлога.
```

### Шта је важно

Први бод: нови смислен гранични тест.

Други бод: образложење прихватања или одбијања предлога.

Одговорност остаје код студента.

## Суштина Вежбе 4

Тестови су дизајнерски сигнал, не само доказ да код тренутно ради.

**Када су границе система добре, тестови постају краћи, јаснији и ближи пословном понашању које студент треба да одбрани.**

---

### КЉУЧНА ПОРУКА

**То је основа за интеграцију и AI workflow.**

## За наредни рад

Припремити пројекат за стабилну интеграцију.

- 01 Написати план тестирања за један use-case пре тест кода.
- 02 Додати успешне, негативне и граничне NUnit тестове.
- 03 Mock користити само на јасним спољним границама.
- 04 Покренути coverage и анализирати једну ризичну грану.
- 05 Означити стабилну верзију језгра када је спремна.